

----- ARCHIVO REGISTRAR.HTML

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Sistema de Asistencia de Alumnos</title>
    <link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhJY6hW+ALEwIH"
  crossorigin="anonymous" />
    <link
      rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.
css" />
    <link rel="stylesheet" href="assets/css/registrar.css" />

    <!--Libreria para alertas-->
    <link rel="stylesheet" href="assets/alerts/css/iziToast.min.css" />
  </head>

  <body>

<!-- Menú Superior Moderno -->


<!-- Cambia la línea del container dentro del nav -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm fixed-top">
  <div class="container-fluid px-5"> <!-- Usamos container-fluid para evitar el
conflicto del bg-white -->
    <a class="navbar-brand fw-bold" href="home.html">
      <i class="bi bi-person-check-fill me-2"></i>Asistencias
    </a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item"><a class="nav-link" href="index.html">Estado</a></li>
        <li class="nav-item"><a class="nav-link"
href="listado.html">Listado</a></li>
```

```

        <li class="nav-item"><a class="nav-link active"
href="Registrar.html">Registrar</a></li>
        <li class="nav-item"><a class="nav-link" href="cursos.html">Cursos</a></li>
        <li class="nav-item"><a class="nav-link" href="ayuda.html">Ayuda</a></li>
    </ul>
    <span class="navbar-text text-white"> <!-- Forzamos texto blanco aquí -->
        <i class="bi bi-person-circle"></i> Admin
    </span>
</div>
</div>
</nav>

```

```

<!-- FIN FIN FIN Menú Superior Moderno -->

```

```

<h1 class="text-center mt-5 mb-5 fw-bold">
    Agregar Alumnos al sistema
    
    <hr />
</h1>

```

```

<div class="container mb-5">
    <div class="row justify-content-md-center">
        <div class="col-md-12">
            <h1 class="text-center">
                <span class="float-start">
                    <a
                        href="#"
                        onclick="window.miModal('agregaralumnoModal','')"
                        class="btn btn-success btn-lg"
                        title="Registrar Nuevo Alumno">
                            <i class="bi bi-person-plus"></i>
                    </a>
                </span>
                Registrar nuevo Alumno
            <hr />
        </h1>
    </div>

```

```

    <div class="table-responsive">
        <table class="table table-hover" id="tablaalumnosSinHuella">
            <thead>
                <tr>
                    <th>Curso</th>
                    <th>Apellido y Nombre</th>
                    <th>DNI</th>
                    <th>Obs</th>
                </tr>
            </thead>
            <tbody></tbody>
        </table>
    </div>

```

```

        </div>
    </div>
</div>

<script src="https://code.jquery.com/jquery-3.7.1.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>

```

```

    <script type="module" src="assets/js/todoCrud.js"></script>
    <script src="assets/alerts/js/iziToast.min.js"></script>
</body>
</html>

```

----- ARCHIVO TODOCRUD.JS

```

-----
import {
    addalumno,
    getalumnosCollection,
    deletealumnoCollection,
    getalumnoCollection,
    updatealumnoCollection,
    marcarParaBorrar,
    getCursosCollection
} from "../firebase.js";

/**
 * Función para refrescar la tabla después de cualquier acción
 */
async function refrescarTabla() {
    await mostraralumnosEnHTML();
}

window.miModal = async function (idModal, idalumno = "") {
    try {
        // 1. Limpieza de seguridad
        const existing = document.getElementById(idModal);
        if (existing) {
            const ins = bootstrap.Modal.getInstance(existing);
            if (ins) ins.dispose();
            existing.remove();
        }
        document.querySelectorAll('.modal-backdrop').forEach(el => el.remove());

        // 2. Cargar el PHP
        let url = "";
        switch (idModal) {
            case "agregaralumnoModal": url = "modales/modalAdd.php"; break;
            case "detallealumnoModal": url = "modales/modalDetalles.php"; break;
            case "editaralumnoModal": url = "modales/modalEditar.php"; break;
            case "eliminaralumnoModal": url = "modales/modalDelete.php"; break;
        }
    }
}

```

```

        default: return;
    }

    const response = await fetch(url);
    const text = await response.text();

    // 3. Convertir texto a elementos y FILTRAR solo el DIV del modal
    const parser = new DOMParser();
    const htmlDoc = parser.parseFromString(text, 'text/html');

    // Esto busca el div.modal real ignorando comentarios y textos
    const modalElement = htmlDoc.querySelector('.modal');

    if (!modalElement) {
        console.error("No se encontró un div con clase 'modal' en el archivo
cargado.");
        return;
    }

    // 4. Inyectar y mostrar
    document.body.appendChild(modalElement);

    const myModal = new bootstrap.Modal(modalElement);
    myModal.show();

    // 5. Cargar datos según corresponda
    if (idModal === "detallealumnoModal") {
        await cargarDetallealumno(idalumno);
    } else if (idModal === "editaralumnoModal") {
        await getalumnoUpdateCollection(idalumno);
    } else if (idModal === "eliminaralumnoModal") {
        const btn = modalElement.querySelector("#confirmDeleteBtn");
        if (btn) btn.onclick = async () => {
            await eliminaralumno(idalumno);
            //await solicitarBorradoHuella(idalumno);
            myModal.hide();
        };
    }

    } catch (error) {
        console.error("Error al abrir modal:", error);
    }
};

// async function mostrarialumnosSinHuella() {
//     try {
//         const queryCollection = await getalumnosCollection();
//         const tablaalumnos = document.querySelector("#tablaalumnos tbody");
//         if (!tablaalumnos) return;

```

```

//      tablaalumnos.innerHTML = "";

//      queryCollection.forEach((doc) => {

//          const alumno = doc.val();

//          const id = doc.key;
//          if (alumno.borrar === "Si") { return; }
//          const fila = document.createElement("tr");
//          fila.id = id;
//          fila.innerHTML = `
//              <td>${alumno.curso}</td>
//              <td>${alumno.nombre}</td>
//              <td>${alumno.dni}</td>
//              <td>${alumno.obs}</td>
//              <td>
//                  <a title="Ver detalles" href="#"
onclick="window.miModal('detallealumnoModal','${id}')" class="btn btn-success
btn-sm">
//                      <i class="bi bi-binoculars"></i>
//                  </a>
//                  <a title="Editar" href="#"
onclick="window.miModal('editaralumnoModal','${id}')" class="btn btn-warning
btn-sm">
//                      <i class="bi bi-pencil-square"></i>
//                  </a>
//                  <a title="Eliminar" href="#"
onclick="window.miModal('eliminaralumnoModal','${id}')" class="btn btn-danger
btn-sm">
//                      <i class="bi bi-trash"></i>
//                  </a>
//              </td>
//          `;
//          tablaalumnos.appendChild(fila);
//      });
//  } catch (error) {
//      console.error("Error al obtener los alumnos:", error);
//  }
// }

/**
 * Función para mostrar alertas (Usando iziToast)
 * Movida arriba para que esté disponible cuando se necesite.
 */
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (typeof iziToast === 'undefined') {
        console.warn("iziToast no está cargado, usando alert normal:", mensaje);
        alert(mensaje);
        return;
    }

```

```

    }

    if (tipoToast === "success") {
        iziToast.success({
            timeout: 5000,
            icon: "bi bi-check-circle-fill",
            title: "¡Éxito!",
            message: mensaje,
            position: "topRight"
        });
    } else if (tipoToast === "error" || tipoToast === "warning") {
        iziToast.error({
            timeout: 5000,
            icon: "bi bi-x-circle-fill",
            title: "¡Error!",
            message: mensaje,
            position: "topRight"
        });
    }
};

// ... resto de tus funciones (addNuevoalumno, etc.)

/**
 * CREATE: Agrega un nuevo alumno
 */
window.addNuevoalumno = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioalumno");
    const formData = new FormData(formulario);
    const dataJSON = Object.fromEntries(formData.entries());

    try {
        await addalumno(dataJSON.curso, dataJSON.nombre, dataJSON.dni,
dataJSON.obs, -1); //-1 sin id de huella asigando
        formulario.reset();

        const modalElt = document.getElementById("agregaralumnoModal");
        bootstrap.Modal.getInstance(modalElt).hide();

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno registrado
con éxito!" });
        await refrescarTabla();
    } catch (error) {
        console.error("Error al añadir:", error);
    }
};

/**

```

```

* READ ONE: Detalles del alumno
*/
async function cargarDetallealumno(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      const contenedor = document.querySelector("#detallealumnoContenido
ul");
      if (!contenedor) return;

      contenedor.innerHTML = `
        <li class="list-group-item"><b>Curso:</b> ${data.curso ||
"N/A"}</li>
        <li class="list-group-item"><b>Apellido y Nombre:</b> ${data.nombre
|| "N/A"}</li>
        <li class="list-group-item"><b>DNI:</b> ${data.dni || "N/A"}</li>
        <li class="list-group-item"><b>Obs:</b> ${data.obs || "N/A"}</li>
      `;
    }
  } catch (error) {
    console.error("Error al cargar detalles:", error);
  }
}

/**
* UPDATE: Cargar datos en el formulario de edición
*/
async function getalumnoUpdateCollection(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      document.querySelector('[name="idalumno"]').value = id;
      document.querySelector('[name="curso"]').value = data.curso;
      document.querySelector('[name="nombre"]').value = data.nombre;
      document.querySelector('[name="dni"]').value = data.dni;
      document.querySelector('[name="obs"]').value = data.obs;
    }
  } catch (error) {
    console.error("Error al obtener datos para editar:", error);
  }
}

/**
* UPDATE: Guardar cambios del alumno
*/
window.actualizaralumno = async function (event) {
  event.preventDefault();
  const formulario = document.querySelector("#formularioalumnoEdit");

```

```

const formData = new FormData(formulario);
const { idalumno, ...datosNuevos } = Object.fromEntries(formData.entries());

try {
  await updatealumnoCollection(idalumno, datosNuevos);

  const modalElt = document.getElementById("editaralumnoModal");
  bootstrap.Modal.getInstance(modalElt).hide();

  window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno
actualizado!" });
  await refrescarTabla();
} catch (error) {
  console.error("Error al actualizar:", error);
}
};

/**
 * DELETE: Lógica principal de borrado
 */

async function eliminaralumno(id) {
  try {
    const snapshot = await getalumnosCollection(); // O la función que uses
    para traer el alumno
    // Nota: Si puedes, usa getalumnoById(id) para ser más eficiente

    // Buscamos los datos del alumno específico en el snapshot o volvemos a
    consultar
    const alumnoSnapshot = await getalumnoCollection(id);
    if (!alumnoSnapshot.exists()) return;
    const alumno = alumnoSnapshot.val();

    if (alumno.huellaId === "-1" || alumno.huellaId === -1) {
      // CASO A: Sin huella -> Borrado físico directo
      await deletealumnoCollection(id);
      window.mostrarAlerta({ tipoToast: "success", mensaje: "Alumno eliminado
correctamente" });
    } else {
      // CASO B: Con huella -> Usamos tu función UPDATE de firebase.js
      // Pasamos el ID y el objeto con el campo que queremos cambiar
      await updatealumnoCollection(id, { borrar: "Si" });

      window.mostrarAlerta({
        tipoToast: "success",
        mensaje: "Enviando orden de borrado al sensor ESP"
      });
    }
  }
}

```



```

        await refrescarTabla();
    } catch (error) {
        console.error("Error en eliminaralumno:", error);
        window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al procesar la solicitud" });
    }
}

```

// Inicialización

```

window.addEventListener("DOMContentLoaded", mostraralumnosEnHTML);

```

```

async function cargarCursosEnSelect() {
    const select = document.getElementById("filtroCurso");
    // Asume que tienes una función getCursosCollection en firebase.js
    const cursosSnap = await getCursosCollection();

    cursosSnap.forEach(doc => {
        const cursoData = doc.val();
        const option = document.createElement("option");
        option.value = cursoData.Curso; // El campo de tu DB
        option.textContent = cursoData.Curso;
        select.appendChild(option);
    });
}

```

```

async function mostraralumnosEnHTML() {
    try {
        const queryCollection = await getalumnosCollection();
        const tablaalumnos = document.querySelector("#tablaalumnos tbody");
        const filtroCurso = document.getElementById("filtroCurso").value;
        const busqueda =
document.getElementById("buscarAlumno").value.toLowerCase();
        const spanContador = document.getElementById("contadorAlumnos"); // El
nuevo span

```

```

        if (!tablaalumnos) return;
        tablaalumnos.innerHTML = "";

```

```

        let contador = 0; // Inicializamos el contador

```

```

        queryCollection.forEach((doc) => {
            const alumno = doc.val();
            const id = doc.key;

```

```

            // 1. Filtro de borrado lógico
            if (alumno.borrar === "Si") return;

```

```

            // 2. Filtro por Curso
            if (filtroCurso && alumno.curso !== filtroCurso) return;

```

```

// 3. Filtro por Apellido o DNI
const coincideNombre = alumno.nombre.toLowerCase().includes(busqueda);
const coincideDNI = alumno.dni.toString().includes(busqueda);
if (busqueda && !coincideNombre && !coincideDNI) return;

// Si llegó aquí, el alumno pasó los filtros
contador++;

// Construcción de la fila
const fila = document.createElement("tr");
fila.id = id;
fila.innerHTML = `
    <td>${alumno.curso}</td>
    <td>${alumno.nombre}</td>
    <td>${alumno.dni}</td>
    <td>${alumno.obs}</td>
    <td>
        <a onclick="window.miModal('detallealumnoModal','${id}')"
class="btn btn-success btn-sm"><i class="bi bi-binoculars"></i></a>
        <a onclick="window.miModal('editaralumnoModal','${id}')"
class="btn btn-warning btn-sm"><i class="bi bi-pencil-square"></i></a>
        <a onclick="window.miModal('eliminaralumnoModal','${id}')"
class="btn btn-danger btn-sm"><i class="bi bi-trash"></i></a>
    </td>`;
    tablaalumnos.appendChild(fila);
});

// Actualizamos el número en el HTML al terminar el bucle
if (spanContador) {
    spanContador.textContent = contador;
}

} catch (error) {
    console.error("Error al obtener alumnos:", error);
}
}

```

```

window.addEventListener("DOMContentLoaded", async () => {
    await cargarCursosEnSelect(); // Carga el select primero
    await mostraralumnosEnHTML(); // Muestra la tabla inicial

```

```

// Eventos de filtrado
document.getElementById("buscarAlumno").addEventListener("input",
mostraralumnosEnHTML);
document.getElementById("filtroCurso").addEventListener("change",

```

```
mostraralumnosEnHTML);
});
```

```
----- ARCHIVOMODALADD.PHP
-----
```

```
<div class="modal fade" id="agregaralumnoModal" data-bs-backdrop="static"
data-bs-keyboard="false" tabindex="-1" aria-labelledby="staticBackdropLabel">
```

```
<div class="modal-dialog modal-dialog-centered">
```

```
<div class="modal-content">
```

```
<div class="modal-header">
```

```
<h1 class="modal-title fs-5 titulo_modal">Registrar Nuevo
```

```
Alumno</h1>
```

```
<button type="button" class="btn-close" data-bs-dismiss="modal"
aria-label="Close"></button>
```

```
</div>
```

```
<div class="modal-body">
```

```
<form id="formularioalumno" action="" method="POST"
autocomplete="off">
```

```
<div class="mb-3">
```

```
<label class="form-label">Seleccione el Curso</label>
```

```
<select name="curso" class="form-select" required>
```

```
<option selected value="">Seleccione</option>
```

```
<?php
```

```
$curso = array(
```

```
    "1ro A",
```

```
    "1ro B",
```

```
    "1ro C",
```

```
    "2do A",
```

```
    "2do B",
```

```
    "2do C"
```

```
);
```

```
foreach ($curso as $curso) {
```

```
    echo "<option value='$curso'$curso</option>";
```

```
}
```

```
?>
```

```
</select>
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label">Apellido y Nombre</label>
```

```
<input type="text" name="nombre" class="form-control"
required />
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label">DNI</label>
```

```
<input type="text" name="dni" class="form-control"
required />
```

```
</div>
```

```

        <div class="mb-3">
            <label class="form-label">Obs</label>
            <input type="text" name="obs" class="form-control"
required />
        </div>

        <div class="d-grid gap-2">
            <button type="submit" class="btn btn-primary btn_add"
onclick="window.addNuevoalumno(event)">
                Registrar nuevo Alumno
            </button>
        </div>
    </form>
</div>
</div>
</div>
</div>
```